

2026년도 미니 캡스톤 프로젝트

머신러닝 데이터 분석 및 예측

프로젝트명	머신러닝기반 태양광 발전량 요인 분석 및 예측 모델 고도화에 필요한 데이터 분석
담당자	김OO

2026년 4 월 3 일

AI개발 수행내용

1. 프로젝트 목표

- 기상 요인과 발전량 간의 상관관계 규명 및 핵심변수 식별
- 비정상 데이터 정제를 통한 예측 데이터셋의 신뢰도 확보
- XGBoost 기반의 고정밀 예측 모델 구축 및 성능 최적화

"본 분석은 정확한 발전량 예측을 통해 이상 기후 징후를 조기에 감지하고, 안정적인 에너지 공급 계획을 수립하는 데 기여 하는 것을 최종 목표로 함."

2. 개요 및 현황

2.1 추진배경 및 목적

- 추진배경 : 태양광 발전은 기상 조건(일사량, 온도 등)에 따른 발전 변동성이 매우 커 정밀한 발전량 예측이 필수적임. 기존 통계 방식의 한계를 극복하고 발전 효율을 극대화하기 위해 AI 기반의 고정밀 예측 모델 도입이 요구됨.
- 목표 : 기상 요인(일사량, 온도)과 발전량 간의 비선형적 상관관계 규명.
XGBoost 알고리즘을 활용한 발전량 예측 오차(RMSE) 최소화 및 예측 신뢰도 확보.
예측 모델을 통한 이상 발전 징후 사전 포착 및 효율적 발전소 운영 근거 마련.

2.2 범위

과제구분			내용
데이터셋	전처리	데이터 통합	발전량 데이터, 날씨데이터를 날짜 기준으로 통합
		Data Cleaning	야간 시간대 및 이상 수치 제거
		Data Scaling	변수 간 단위 표준화 (StandardScaler)
	EDA 분석	분포 분석	변수별 수렴도 및 편향성 확인 (hist)
		상관성 분석	피어슨 상관계수 산출 및 다중공선성 진단 (heatmap)
		결측치 확인	데이터 완전성 검토 및 삭제/대치 처리
		시각화 분석	주요 변수 간의 관계 시각화 (Pairplot)
			일사량-발전량 간 선형 관계성 분석 (Lmplot)
			평균 발전량 분석 (Lmplot)
			한 달간의 일일 총 발전량 추이 분석 (Lmplot)
Ai 머신러닝	모델 구축		비선형 패턴 학습 최적화 모델 채택(XGBoost)
	학습 및 예측		분석 변수 선택, 데이터 분리, 예측 및 결과 출력
	분석	오차 분석	Actual vs Predicted 차이 분석 (scatter)
		중요도 분석	Feature Importance 산출물 기여도 인자 분석(barplot)
	모델 성능평가	지표 계산	mae, mse, rmse, r2 기반 모델 신뢰도 검증
	프로토타이핑		Streamlit을 이용한 예측 웹 개발

2.3 과제 추진 방법

2.3.1 구축 대상 선정 기준

- 데이터 완전성 : 결측치가 없고, 발전 데이터와 기상 데이터의 타임스탬프(DATE_TIME)가 일치하는 구간을 우선 선정.
- 유의미성 : 상관관계 분석 결과, 발전량과 높은 양(+)의 상관관계를 보이는 일사량(IRRADIATION > 0) 구간을 주요 분석 대상으로 선정하여 예측 정확도를 높임.

2.3.2 AI 예측 분석모델 적용 대상

환경관리 기능	수집 데이터	독립변수	종속변수
하수	- Plant_1_발전량 데이터 (Generation Data) - Plant_2_날씨 기후 데이터 (Weather Sensor Data)	- 일사량: 발전의 주 에너지원으로 가장 핵심적인 예측 인자. - 모듈 온도: 패널 과열에 따른 효율 저하 패턴을 학습하기 위한 필수 인자. - 주변 온도: 외부 환경 기온에 따른 간접적 효율 변동성 반영.	- 인버터 전 단계의 순수 전력 생산량인 DC 발전량 (DC_POWER).

2.3.3 AI 분석모델 구축 프로세스

[1단계] 데이터 수집 및 통합

발전량(Generation) & 기상(Weather) 데이터 로드

[2단계] 탐색적 데이터 분석 (EDA)

변수별 분포 및 상관관계(Heatmap) 분석
핵심 변수(일사량-발전량) 간 선형 관계 확인

[3단계] 데이터 전처리 (Preprocessing)

야간 시간대(Irradiation=0) 데이터 필터링
논리적 오류치 제거 및 핵심 변수 컬럼 정제
변수 간 단위 통일을 위한 StandardScaler 적용

[4단계] AI 모델 학습 (Machine Learning)

데이터셋 분리 (훈련 8 : 테스트 2)
비선형 패턴 학습에 강한 XGBoost 모델 학습

[5단계] 성능평가 및 분석

예측 정확도(R², RMSE, MAE) 지표, 산점도 및 Feature Importance 분석

[Feedback Loop] 오차가 클 경우, 화살표를 다시 [2단계] or [3단계]로 연결하여 변수 재설정 권장



분석 내용

1. 데이터 수집

1-1. Plant_1_Generation_Data

○ 캐글 solar_data by samuel Waweru

	A	B	C	D	E	F	G
1	DATE_TIME	PLANT_ID	SOURCE_KEY	DC_POWER	AC_POWER	DAILY_YIELD	TOTAL_YIELD
44158	06-06-2020 07:00	4135001	3PZuoBAID5Wc2HD	2888.875	283.5375	110.875	7152596.88
44159	06-06-2020 07:00	4135001	7JYdWkrLSPkdwr4	2741.75	269.0875	101.875	7764241.88

DATE_TIME : 일시

PLANT_ID : 발전소 ID

SOURCE_KEY : 인버터 ID - 특정 인버터나 센서를 구분하는 고유 키

DC_POWER : 생성된 직류 전력

AC_POWER : 인버터를 거쳐 교류로 변환된 전력

DAILY_YIELD : 일일 발전량

TOTAL_TIELD : 총 누적 발전량

1-2. Plant_2_Weather_Sensor_Data

○ 캐글 solar_data by samuel Waweru

	A	B	C	D	E	F
1	DATE_TIME	PLANT_ID	SOURCE_KEY	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION
3120	2020-06-17 8:00	4135001	HmiyD2TTLFNqkNe	23.751955	31.0847412	0.234509729
3121	2020-06-17 8:15	4135001	HmiyD2TTLFNqkNe	24.041337	31.60524687	0.261087749

AMBIENT_TEMPERATURE : 주변기온 - 실외 온도

MODULE_TEMPERATURE : 태양광 패널 표면 온도

IRRADIATION : 일사량

2. 데이터 통합

1-1. 데이터 스케일링

○ DATE_TIME 형식 통일

데이터 통합

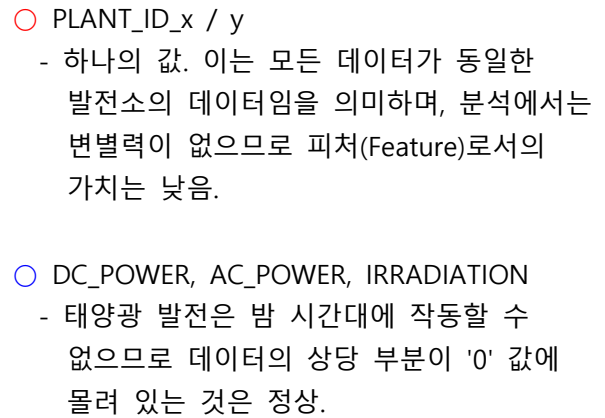
```
# 1. 데이터 불러오기
gen = pd.read_csv('./dataset/Plant_1_Generation_Data.csv')
weather = pd.read_csv('./dataset/Plant_2_Weather_Sensor_Data.csv')

# 2. 날짜 형식 변환
gen['DATE_TIME'] = pd.to_datetime(gen['DATE_TIME'], dayfirst=True)
weather['DATE_TIME'] = pd.to_datetime(weather['DATE_TIME'])

# 3. 데이터셋 합치기
df = pd.merge(gen, weather, on='DATE_TIME', how='inner')
```

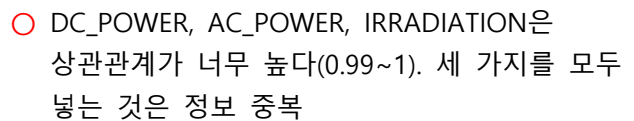
	A	B	C	D	E	F
1	DATE_TIME	DC_POWER	AC_POWER	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION
38237	2020-06-17 18:30	79	7.61428571	23.8402385	22.96865843	0.00700688981
38238	2020-06-17 18:30	75.7142857	7.3	23.8402385	22.96865843	0.00700688981

3-1. 변수별 데이터 분포 확인



- 변수 제거(Feature Selection) : PLANT_ID_x, PLANT_ID_y처럼 값이 하나뿐인 변수들 제거
- 밤 데이터 제거

[다중공선성]



- 값이 하나뿐이라 계산이 안 된 것입니다.
분석에서 제외
- TOTAL_YIELD 역시 현재 시점의 발전량 예측에는 큰 도움이 되지 않는 데이터이므로 제외

○ 발전량을 예측할 때 가장 중요한 외부 요인은 IRRADIATION(일사량)과 DULE_TEMPERATURE(패널 온도)이다. 두 데이터는 핵심 독립 변수로 사용.

3-3. 밤 데이터 제거 후 결측치 확인

```
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DATE_TIME              3182 non-null    object
1   PLANT_ID               3182 non-null    int64
2   SOURCE_KEY             3182 non-null    object
3   AMBIENT_TEMPERATURE    3182 non-null    float64
4   MODULE_TEMPERATURE     3182 non-null    float64
5   IRRADIATION            3182 non-null    float64
dtypes: float64(3), int64(1), object(2)
memory usage: 149.3+ KB
```

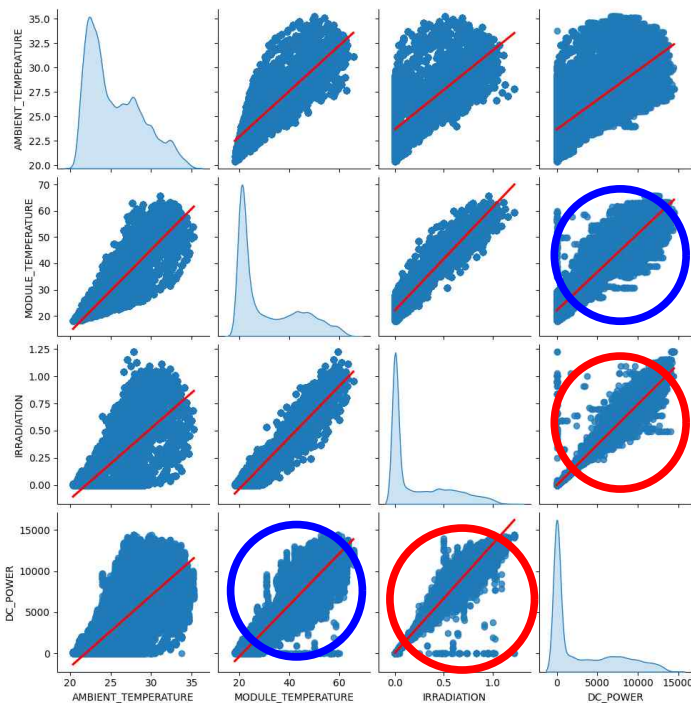
전체 행(RangeIndex) : 3182개

Non-Null Count : 3182개

[결론]

평균치입력 or 추론 작업 불필요

3-4. Pairplot : 주요 변수들 간의 관계 시각화 (점의 밀집도)



○ IRRADIATION vs DC_POWER

빨간 선 주변에 파란 점들이 밀집하게 붙어 있다. 즉 일사량만 알면 발전량을 거의 정확하게 맞출 수 있다는 뜻으로 예측 모델을 만들 때 일사량이 가장 중요함

○ MODULE_TEMPERATURE vs DC_POWER

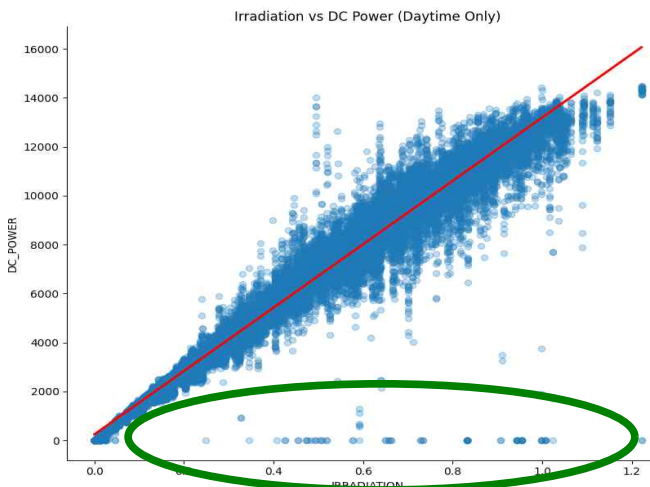
패널 온도와 발전량도 비례하지만, 일사량보다는 점들이 조금 더 퍼져 있습니다. 이는 온도가 발전량에 영향을 주긴 하지만, 일사량만큼 직접적이지는 않다.

[Pairplot 결론]

○ 변수 선택

IRRADIATION과 MODULE_TEMPERATURE 를 주력 독립 변수로 채택

3-5. Lmplot (Linear Model plot) : 주요 변수 관계 시각화



○ 이상치 발견

바닥(0)에 붙어 있는 점, 특히 IRRADIATION(일사량)은 0.5가 넘어가는데 DC_POWER가 0인 점

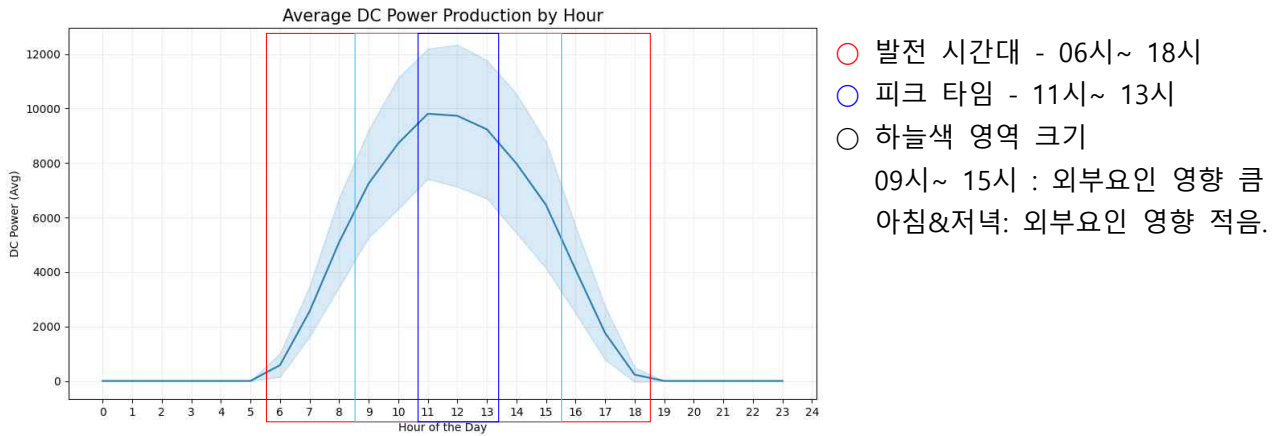
해석 : 일사량은 충분한데(0.4~1.2) 발전량이 0
→ 인버터 고장, 센서 오류, 유지보수 중, 일시적 Cloud shading.

[Lmplot 결론]

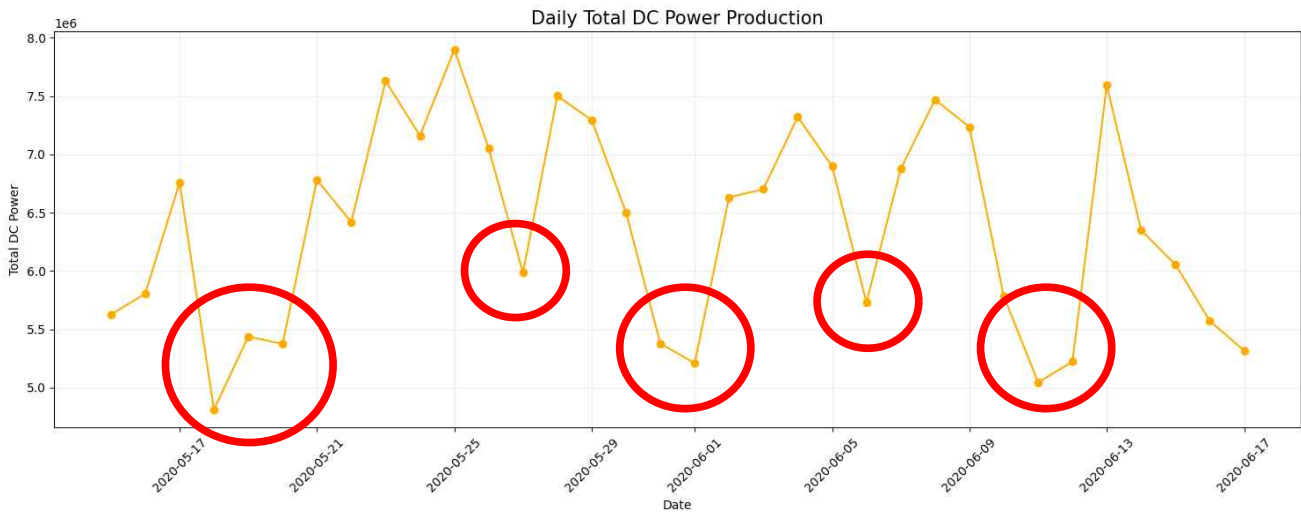
○ 이상치 점들을 비정상 데이터로 간주.

제거하고 학습시켜야 모델의 정확도가 높아짐

3-6. 시간대별(Hour) 평균 발전량 분석 lineplot



3-7. 일자별 발전량 합계 계산, 한 달간의 일일 총 발전량 추이



◎ 변동성이 매우 큼 : 태양광 발전이 외부 환경에 민감하다.

◎ 이상치 후보 포착 : 단순히 날씨가 계속 흐려진 것일 수도 있지만, 만약 기상 데이터와 대조했을 때 날씨가 맑았는데도 이렇다면 패널의 오염(먼지)이나 장비 효율 저하를 의심해 볼 수 있는 구간.

3.8. EDA 결론

[EDA 총 결론]

○ 데이터 정제 (Data Cleaning)

- 이상치 제거 : $(IRRADIATION > 0.1) \ \& \ (DC_POWER == 0)$
- 밤 데이터 처리

○ 제거 컬럼 선정

- 'PLANT_ID_x', 'PLANT_ID_y', 'SOURCE_KEY_x', 'SOURCE_KEY_y'

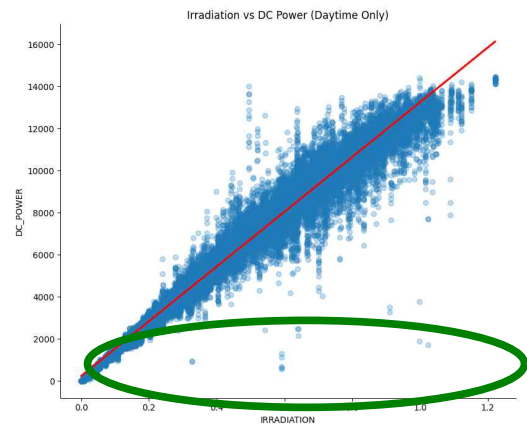
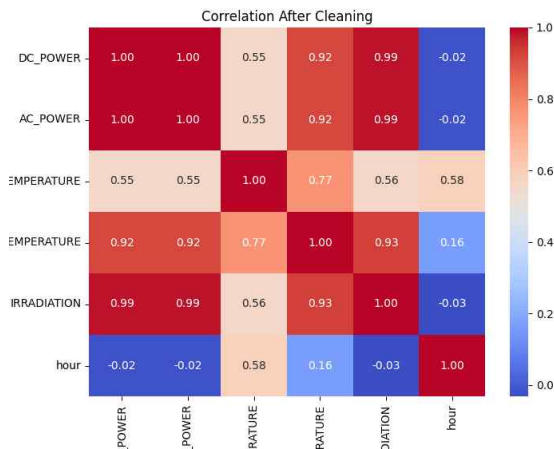
4. 데이터 전처리

4-1. EDA 분석에 따른 데이터 전처리

데이터 전처리

```
# 이상치 제거 : 밤 시간대 데이터 제거
df_clean = df[df['IRRADIATION'] > 0].copy()
# 논리 오류 제거 : 일사량은 충분한데 발전량 0 데이터 제거
df_clean = df_clean[~((df_clean['IRRADIATION'] > 0.01) & (df_clean['DC_POWER'] == 0))]
# 불필요 컬럼 제거
drop_cols = ['PLANT_ID_x', 'PLANT_ID_y', 'SOURCE_KEY_x', 'SOURCE_KEY_y']
df_clean = df_clean.drop(columns=[col for col in drop_cols if col in df_clean.columns])
# 인덱스 초기화
df_clean = df_clean.reset_index(drop=True)
```

4-2. 클리닝 후 heatmap, 산점도 확인



- 클리닝 후 heatmap : 불필요데이터 삭제. 상관관계가 더 순수하게 측정됨. 보통은 상관계수 상승.
- 클리닝 후 산점도 : 논리적 오류 삭제

[클리닝 기대 효과]

- 머신러닝 모델 정확도 상승

5. 머신 러닝 - XGBoost

머신 러닝

```
# 변수 선택
X = df_clean['IRRADIATION', 'AMBIENT_TEMPERATURE', 'MODULE_TEMPERATURE']
y = df_clean['DC_POWER']

# 데이터 분리 (8:2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

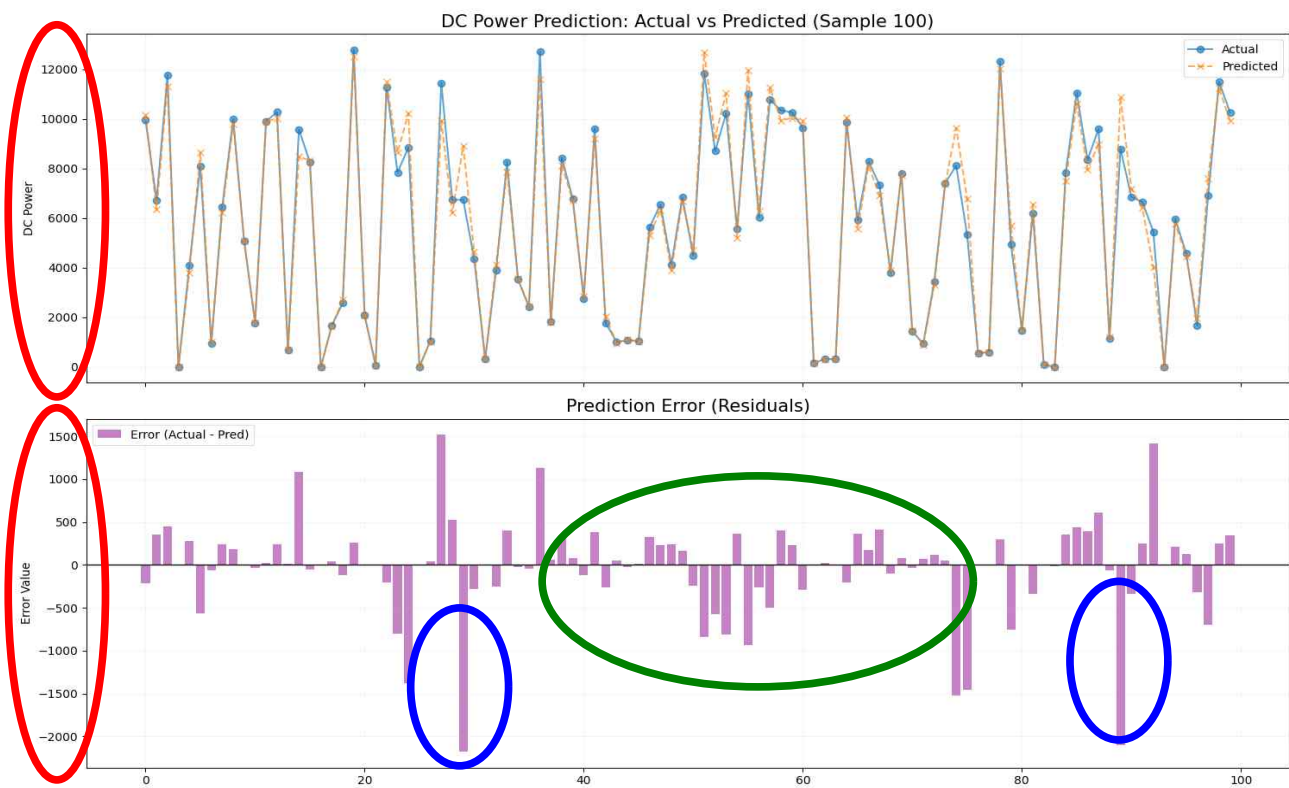
# 데이터 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 모델 선정, 모델 학습
model = xgb.XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    random_state=42)
model.fit(X_train_scaled, y_train)

# 예측 및 결과 출력
y_pred = model.predict(X_test_scaled)
```

6. 분석

6-1. Scatter Plot : Actual vs Predicted



○ 오차의 규모(Vertical Scale)

DC Power의 범위가 0~12,000에 비해, 대부분의 오차가±500 안팎임. 이는 모델이 약 95% 이상의 높은 정확도를 유지하고 있음을 의미함

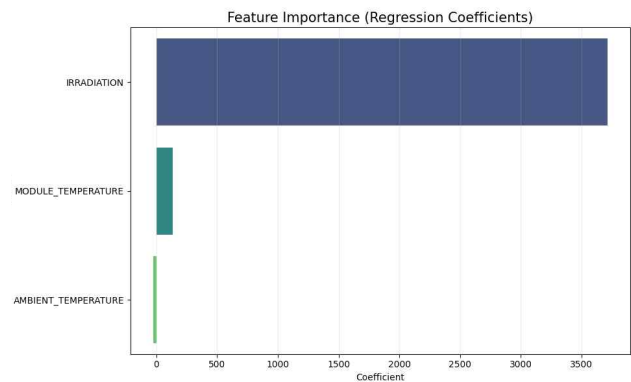
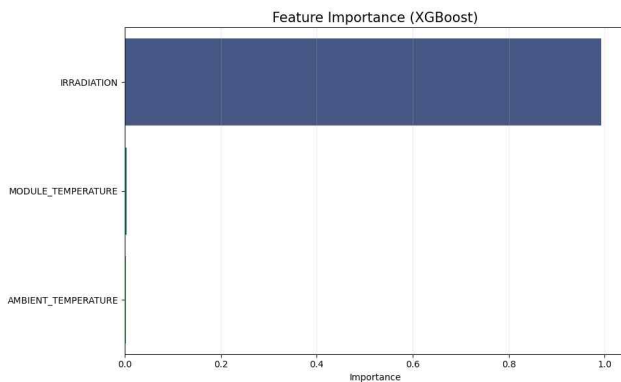
○ 음수 긴 막대 (Over-predicted)

음수 오차는 모델이 실제보다 높게 예측했다는 뜻, 일사량은 충분한데, 실제 발전량은 기대보다 낮음. 즉, 구름이 아주 잠깐 가렸거나, 패널이 일시적으로 과열되어 효율이 떨어진 '순간적인 노이즈' 구간일 가능성이 큼

○ 오차의 불규칙성(Randomness) - Bias

다행히 오차가 특정 방향으로만 쏠리지 않고 위아래로 골고루 퍼져 있습니다. 이는 모델에 편향(Bias)이 없다는 좋은 뜻.

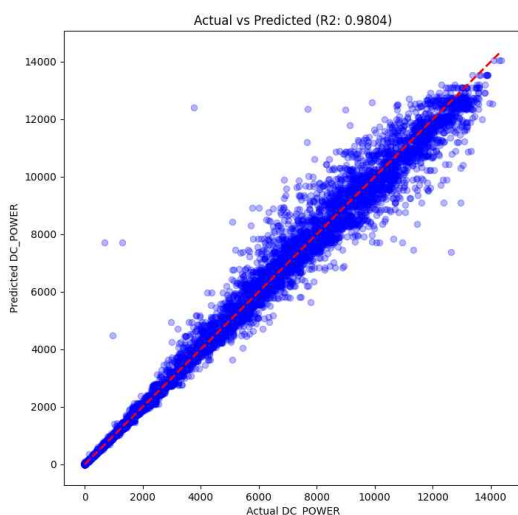
6-2. 특성 중요도



IRRADIATION (일사량)이 발전량에 가장 결정적인 양(+)의 영향을 미침
즉, 빛이 강할수록 발전량이 선형적으로 크게 증가

7. 모델 성능평가

7-1. 분포도 Actual vs Predicted



[분포도 결과]

고출력 구간에서 약간의 변동성이 있지만, 전반적으로 매우 안정적임.

외톨이 점들이 너무 많다면 데이터 클리닝을 더 정교하게 해야 하지만, 지금은 전체 데이터 양에 비해 소수라 무시해도 좋은 수준임.

7-2. 지표 계산

지표

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

	XGBoost	linear_model
MAE (평균 절대 오차)	321.03	423.98
MSE	296108.93	426369.88
RMSE	544.16	652.97
R2 Score (결정 계수)	0.9804	0.9718

- R2 Score (0.9804) : 일사량과 온도가 발전량을 결정짓는 핵심 지표임을 다시 한번 증명
- MAE (321.03) : 예측값과 실제값 사이에 평균적으로 약 321kW 정도의 오차가 발생
- RMSE (544.16) : MAE와 RMSE의 차이가 안정적이다(321-544). 모델이 특정 시간대나 특정 기상 조건에서 터무니없는 예측값을 내뱉는 '대형 사고'를 치지 않고 고르게 잘 맞춤

8. 결론

- 데이터 인사이트(EDA 결과) 핵심 변수 식별 : 태양광 발전량(DC_POWER) 결정의 압도적인 요인은 일사량(IRRADIATION)임을 확인
- 물리적 상관관계 검증: 일사량이 증가함에 따라 모듈 온도(MODULE_TEMPERATURE)가 급격히 상승하며, 이는 발전량과 양(+)의 상관관계를 가짐을 시각적으로 증명
- 이상치 처리의 중요성 : 일사량은 존재하나 발전량이 0인 '장비 고장/인버터 셧다운' 데이터를 전처리 단계에서 제거함으로써 모델의 신뢰도를 높임
- 모델 성능 비교 : 선형 회귀(R2=0.9718)도 우수한 성적을 보였으나, 기상 조건의 미세한 비선형적 변화를 완벽히 포착하기에는 RMSE(652.97)가 다소 높게 나타남. XGBoost 도입 후 R2가 0.9804로 상승했으며, 특히RMSE가 약17% 감소(652 → 544)함. 이는 트리 기반 모델이 온도와 일사량 간의 복잡한 상호작용을 더 정교하게 학습했음을 의미함.
요약 : 전처리된 기상 데이터를 바탕으로 XGBoost 모델을 구축하여, 발전량 예측 오차를 최소화하고 98% 이상의 설명력을 확보함.

9. 향후 발전 계획

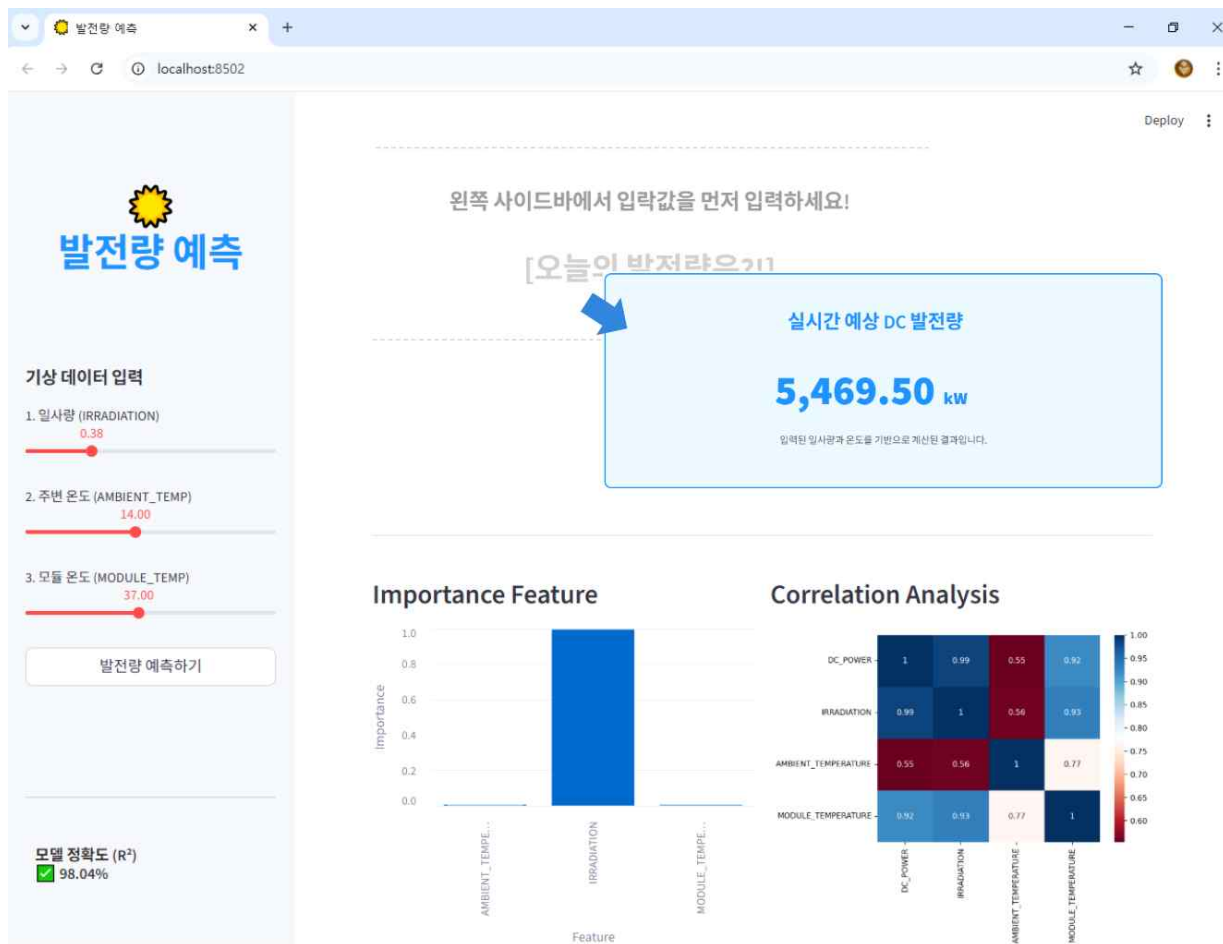
- 피처 엔지니어링 고도화(Feature Engineering)
 - 파생 변수 생성 : 단순 온도 데이터 대신, 온도의 차이(Delta T) 변수를 추가하여 패널의 냉각 효율이 발전량에 미치는 영향을 심도 있게 분석 가능.
 - 시간적 특성 반영 : hour 대신, 시간의 주기성(Sine/Cosine Transformation)을 반영하여 일출/일몰 전후의 급격한 발전량 변화를 더 정확히 예측 가능.
- 딥러닝 모델 도입(LSTM/GRU)
 - 태양광 발전은 직전 시간의 발전량이 다음 시간에도 영향을 주는 시계열(Time-Series) 특성이 강함. 단순 회귀를 넘어 LSTM(Long Short-Term Memory) 등 순환 신경망을 도입하여 시간 흐름에 따른 변동성을 학습시켜 성능 향상 가능.

○ 실무 적용 및 유지보수 시나리오

- 고장 감지 시스템 : 모델의 예측값과 실제 발전량의 차이(오차)가 일정 수준(예: 3 * RMSE)을 넘어서는 경우, 이를 장비 이상 신호로 간주하여 관리자에게 알림을 보내는 시스템으로 확장 가능합니다.
- 기상 예보 데이터 연동 : 과거 데이터 학습에 그치지 않고, 기상청 API를 연동하여 내일의 예상 발전량을 미리 예보하는 서비스로 발전시킬 수 있습니다.

10. 프로토타이핑

Streamlit 배포화면



11. 후기

○ 분석 목적에 따른 최적의 모델 선정 안목 배양

- 단순히 예측 성능(R2 Score)만 따진다면 XGBoost가 압도적이었으나, **각 독립변수(일사량, 온도 등)가 결과값에 미치는 영향력**을 정량적으로 파악하는 데는 선형 회귀(Linear Regression) 모델의 해석력이 월등함을 확인했습니다. 이를 통해 무조건 고성능 모델을 고집하기보다, '설명 가능한 AI'가 필요한지 혹은 '정밀한 예측'이 필요한지 등 비즈니스 **상황과 목적에 부합하는 알고리즘을 선택하는 안목**이 엔지니어의 핵심 역량임을 깨달았습니다.

○ 데이터 전처리(EDA) 및 스케일링의 실질적 효용 체감

- 데이터 클리닝 과정을 통해 노이즈를 제거하는 것만으로도 모델의 전체적인 성능이 비약적으로 향상되는 것을 목격하며 EDA(탐색적 데이터 분석)의 중요성을 깊이 체감했습니다. 특히 트리 기반 모델인 XGBoost조차 적절한 데이터 스케일링을 거쳤을 때 안정적인 학습이 가능함을 확인하며, **정해진 공식에 의존하기보다 데이터의 특성에 맞춰 유연하게 대응해야** 한다는 교훈을 얻었습니다.

○ 데이터 기반의 추론과 가치 창출 역량의 중요성

- 동일한 데이터셋이라 할지라도 분석가의 역량에 따라 도출할 수 있는 인사이트의 깊이가 달라질 수 있음을 실감했습니다. 데이터를 단순히 나열하는 것에 그치지 않고, 그 속에 숨겨진 상관관계를 추론하고 시스템의 가치로 전환하는 과정이 곧 개발자의 차별화된 경쟁력임을 배웠습니다. 이번 프로젝트는 'Data-Driven Developer'로서 한 단계 성장하는 중요한 계기가 되었습니다.